Colab Link: ∞ **B22EE088_B22ES003.ipynb**

Name	Mahek Vanjani, Pranav Bhansali
Roll No	B22EE088, B22ES003
Assignment	04

Deep Learning Assignment-4

Data-Free Adversarial Knowledge Distillation (AKD)

Objective:

The objective of this project was to implement and evaluate a **Data-Free Knowledge Distillation** technique, specifically Adversarial Knowledge Distillation (AKD), for training a compact student CNN model on the CIFAR-100 dataset. The core constraint was to perform this training ***without access to the original CIFAR-100 training images***. A **pre-trained ResNet-34** model served as the teacher, providing supervision through its responses to images generated by a concurrently trained generator network. The script was designed to allow experimentation with different student model architectures.

Dataset:

We use the **CIFAR-100** dataset for evaluation purposes only. The CIFAR-100 dataset consists of

- 50,000 training images (not used in this work)
- 10,000 test images used only for evaluating student performance
- 100 classes of 32×32 color images, with 600 images per class

Two test splits are considered:

- 20% of the test set (~2000 images)
- 10% of the test set (~1000 images)

Dataset Usage:

1. **Training:** The CIFAR-100 **training set was explicitly NOT used** for student model training, adhering to the data-free constraint.
2. **Evaluation:** The standard CIFAR-100 **test set (10,000 images)** was used exclusively for evaluating the trained student model's performance after each epoch. Evaluation metrics were calculated on the full test set.

Dataset Preprocessing:

Standard normalization specific to CIFAR-100 (mean=[0.5071, 0.4867, 0.4408], std=[0.2675,

0.2565, 0.2761]) was applied *only* to the real test images during evaluation. Generated images were used directly by the models during training without explicit normalization.

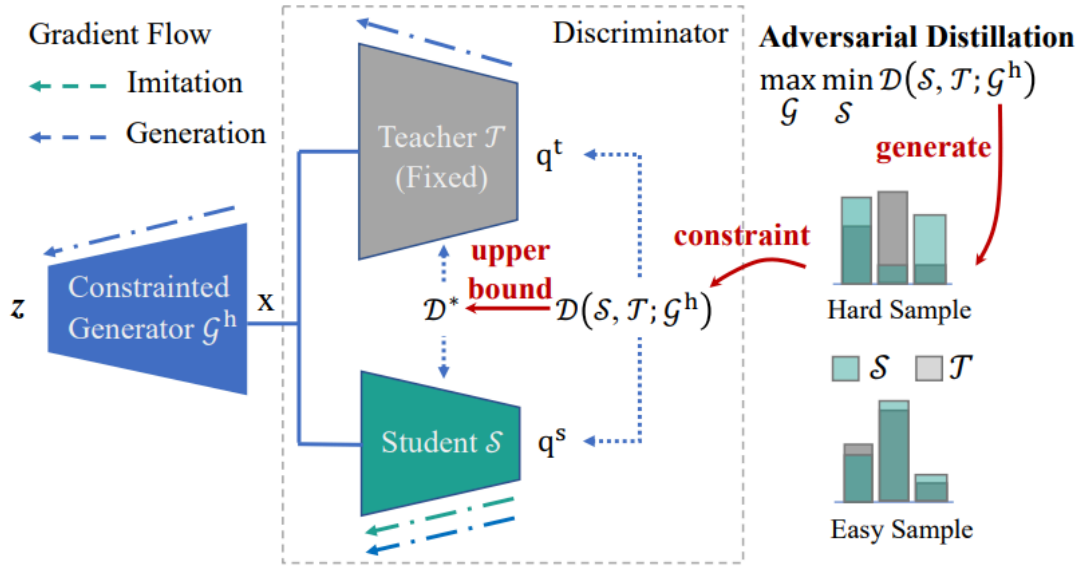


Figure 2: Framework of Data-Free Adversarial Distillation.

Model Explanation:
The figure below shows the basic flow of this model:

- **Teacher Model:**
  - **Architecture:** ResNet-34 (using the provided ResNet class definition adapted for 32x32 inputs).
  - **Weights:** Loaded from a pre-trained checkpoint file.
  - **Role:** Acts as a fixed source of knowledge. Its parameters were frozen during student training. It provides target logits (t_logits) based on its predictions on generated images.
  - **Parameters:** 21328292

- **Student Model:**

    - **Architecture:** Dynamically selected via the --student_model_arch argument. The script provides implementations for:
        - ResNet18_8x: Standard ResNet-18 (~11.1M params, ~52% of ResNet-34).
        - ResNet26_8x: Custom ResNet ([3,3,3,3] blocks , ~ 20% of ResNet-34).
    - **Role:** The model being trained. It learns to mimic the teacher's logit outputs on the images produced by the generator.
    - We also tried for 10% and 20% parameters of the teacher model, but the accuracy and performance were even worse.
    - Also,We tried for different custom CNN models but it was not giving appropriate results.
- **Generator Model:**
    - **Architecture:** GeneratorA (specified as GeneratorA in the configuration). A convolutional generator network takes a latent vector (nz=256) as input and produces a 3x32x32 image with pixel values in the range [-1, 1] (via Tanh activation). It uses Upsampling, Conv2D, BatchNorm, and LeakyReLU layers.
    - **Role:** Synthesizes image data (fake_images) used as input for both the teacher and student during the distillation process. It learns adversarially to generate images that facilitate knowledge transfer.

Loss Function Design:

The success of the Adversarial Knowledge Distillation (AKD) framework critically depends on the careful formulation of loss functions that guide both the student and the generator. In this project, the losses are designed to enable effective knowledge transfer in a data-free setting:

1. Student Loss—Imitation Objective

To ensure that the student mimics the behavior of the pre-trained teacher, we use an **L1 loss (Mean Absolute Error)** between the output logits of the student and the teacher. This objective encourages the student to closely replicate the teacher's soft outputs on the synthetic data generated by the generator.

$$\mathcal{L}_S = \frac{1}{N} \sum_{i-1}^{N} \left\| s\_logits^{(i)} - t\_logits^{(i)} \right\|_1$$

Where:

- s_logits(i) are the student logits for the iii-th synthetic image.

- t_logits(i) are the corresponding teacher logits.
- N is the batch size.

This loss is minimized during the **Student Imitation Phase**, and it drives the student's weights to align its predictions with the teacher's output distribution, even in the absence of real data.

2. Generator Loss—Adversarial Objective

The generator is trained adversarially, with the goal of producing synthetic images that **maximize** the discrepancy between the teacher and student outputs. This is formulated by **negating the student's imitation loss**, effectively encouraging the generator to find regions of the input space where the student fails to generalize well.

$$\mathcal{L}_G = -\mathcal{L}_S = -\frac{1}{N}\sum_{i-1}^{N}\left\|s\_logits^{(i)} - t\_logits^{(i)}\right\|_1$$

By minimizing this loss, the generator is incentivized to synthesize more challenging, diverse, and informative samples that expose weaknesses in the student's current representation capacity.

3. Combined Training Dynamics

The interaction between these two losses forms the core of the adversarial training loop:

- The **student** minimizes LS to better match the teacher.
- The **generator** minimizes LG to fool the student by generating samples it struggles with.
- This push-pull dynamic creates a feedback loop that continuously improves both the generator's sample quality and the student's generalization ability.

Approach: Adversarial Knowledge Distillation (AKD)

The training process adopted in this project closely mirrors the structure of the reference Data-Free Adversarial Distillation (DFAD) framework. The overall methodology is based on **iterative, adversarial training** involving two key components: a generator and a student network, both interacting under the supervision of a frozen teacher network (ResNet-34). The training proceeds in an **alternating fashion** across two distinct phases within each epoch.

1. Iterative Training Structure

For every epoch, the training is divided into multiple sub-iterations, controlled by a hyperparameter iterations_per_epoch. Within each sub-iteration, the student and generator are trained alternately in two phases:

2. Student Imitation Phase

This phase focuses on training the student network to mimic the behavior of the teacher on synthetically generated data. It is performed for k_imit_steps (default = 5) inner iterations for every single generator update. The steps involved are

- **Synthetic Data Generation:**
  The generator creates a batch of fake images from random noise. These images are unnormalized and treated as raw synthetic inputs.
- **Logit Extraction:**
  - The **student model** processes the generated images to produce output logits s_logits.
  - The **teacher model** (which is frozen and pre-trained on CIFAR-100) processes the same batch of images to produce its logits t_logits.
- **Loss Computation:**
  The student aims to minimize the discrepancy between its predictions and those of the teacher. This is achieved using **L1 loss** (mean absolute error) between the student's and teacher's logits:
  $$\mathcal{L}_S = \|s\_logits - t\_logits\|_1$$
- **Student Update:**
  The gradients are computed with respect to this loss, and the student model's weights are updated accordingly using backpropagation.

3. Generator Adversarial Phase

Following the student update steps, the generator is trained to improve its ability to challenge the student by synthesizing images that the student struggles to mimic accurately. This phase is executed once per outer iteration.

- **Image Generation:**
  The generator again creates a fresh batch of fake images.

- **Logit Comparison:**
  Both the student and teacher networks process these images to generate their respective logits (s_logits and t_logits).
- **Adversarial Loss Computation:**
  The generator is trained to **maximize** the L1 difference between the student's and teacher's logits — effectively seeking to produce images that the student cannot imitate well. This is implemented by **minimizing the negative L1 loss**:

$$\mathcal{L}_G = -\|s\_logits - t\_logits\|_1$$

- **Generator Update:**
  The gradients from this adversarial loss are used to update the generator's parameters, enabling it to create more informative and challenging synthetic samples in future iterations.

Training and Hyperparameters:

1. **Student Architecture:**
2. Epochs: 50
3. Optimizer (Student): SGD (LR: 0.1, Momentum: 0.9, Weight Decay: 5e-4)
4. Optimizer (Generator): Adam (LR: 1e-4, Betas: (0.5, 0.999))
5. LR Scheduler (Student): MultiStepLR (Milestones: [25, 40], Gamma: 0.1)
6. Batch Size: 256 (Train), 256 (Test)
7. Iterations per Epoch: 100
8. Student Steps (k_imit_steps): 5
9. Latent Dimension (nz): 256
10. Seed: 42

Additional Configurations:

- Dataset: CIFAR-100
- Image Size: 32x32
- Input Channels: 3
- Number of Classes: 100
- Teacher Weights Path: /content/best_resnet34_cifar100.pth
- Checkpoint Directory: ./checkpoints_akd_gp_cifar100
- Save Prefix: akd_gp_cifar100
- Log Interval: Every 20 iterations
- Parameter Tolerance for Student: ±3%

Evaluation and Results:
Performance metrics:
1. **Metrics Evaluated:**

- **Test Accuracy:** Measures the percentage of correctly classified test samples.
- **Average Cross-Entropy Loss:** Represents the mean loss between predicted and true class probabilities, indicating model confidence and correctness.

2. Procedure:

After each training epoch, the current student model was evaluated on the full CIFAR-100 test set using standard test-time transformations, including normalization.

3. Checkpointing:

The student model and its corresponding generator achieving the highest test accuracy throughout training were saved. The final model's performance and best accuracy achieved during training are reported below.

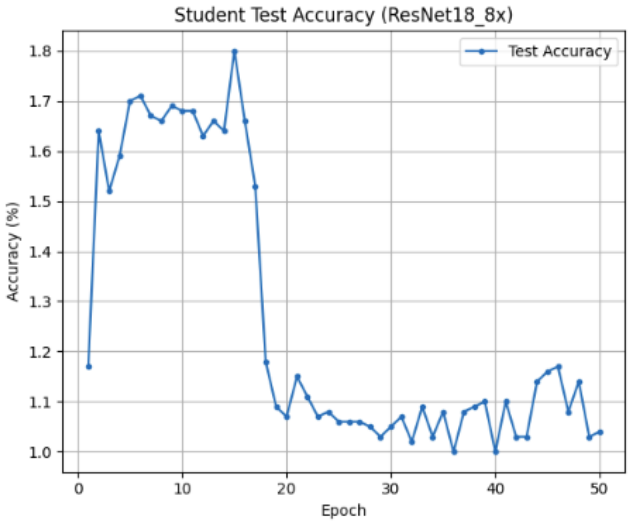
4. Results:
For a 10%-90% train-test split:

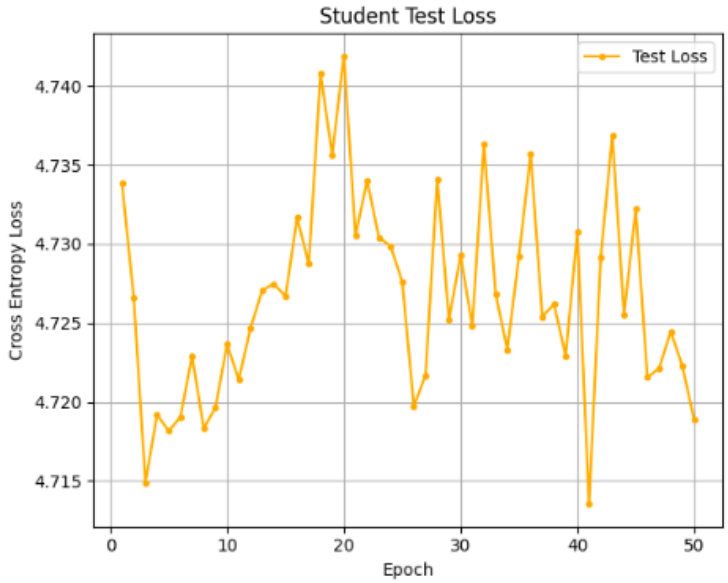
Metric	Value
Final Test Accuracy	1.06%
Best Test Accuracy	1.82%
Final Test Loss	4.7189
Best Checkpoint Saved As	`akd_cifar100_ResNet18_9x`
Total Training Time	1 hour 52 minutes 9.65 seconds

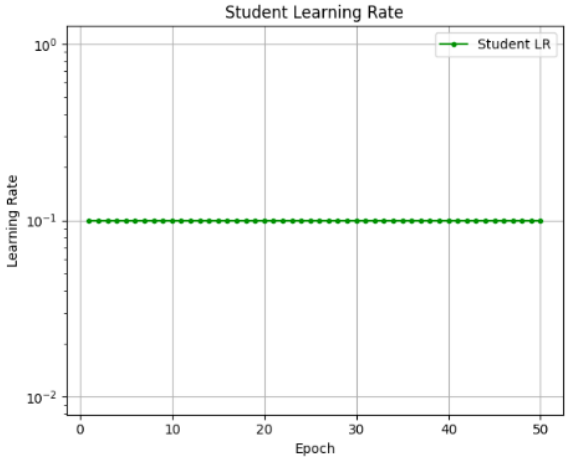
For a 20%-80% train-test split:

Metric	Value
Final Test Accuracy	1.04%
Best Test Accuracy	1.80%
Final Test Loss	4.7189
Best Checkpoint Saved As	`akd_cifar100_ResNet18_8x`
Total Training Time	1 hour 52 minutes 9.65 seconds

5. Plots:

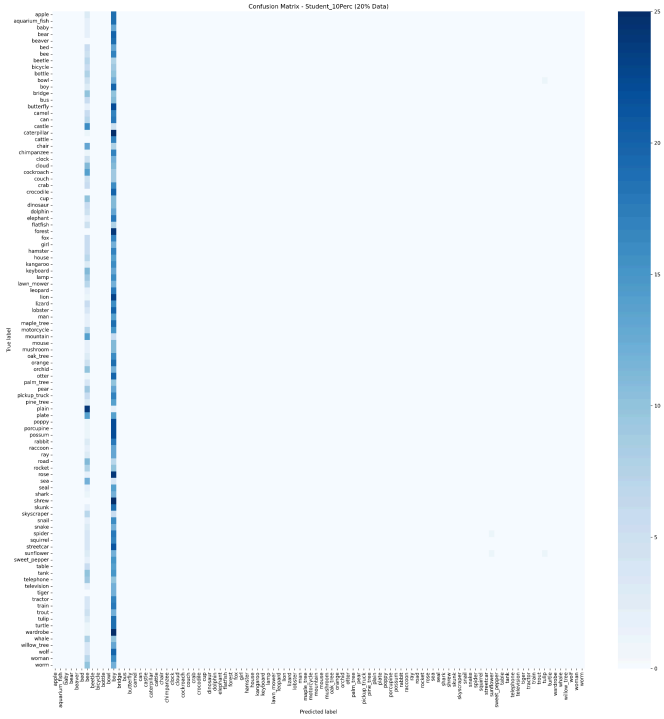




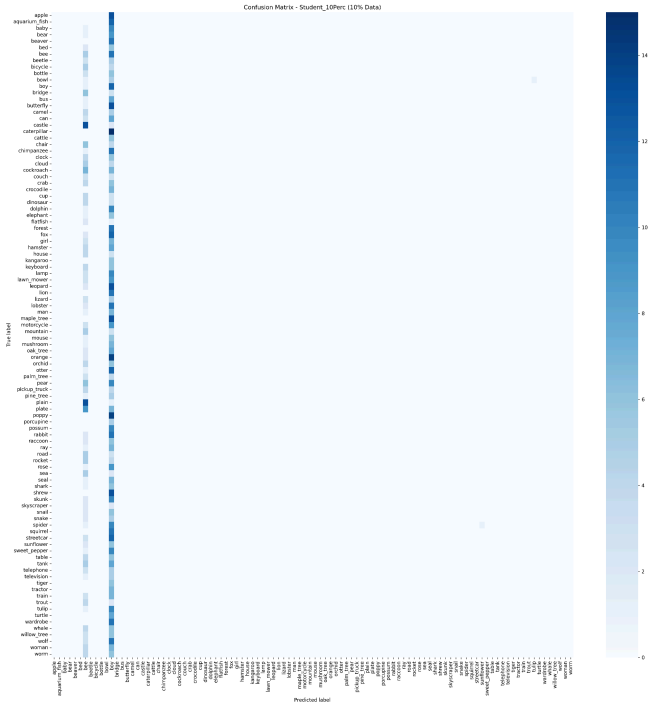


Confusion Matrix:

For 50% of teacher parameters:under 80/20 split



Under 90/10 split:



Discussion:

1. Mode Collapse: The generator suffered from **mode collapse**, producing limited variations of samples. This significantly degraded the student's ability to generalize, as it relied on low-diversity data. Key consequences:

- Synthetic images lacked inter-class and intra-class variance.
- The student received redundant or uninformative supervision.
- Accuracy remained stagnant after initial epochs.

2. Teacher-Student Resolution Mismatch

The **teacher model was trained with 224×224 resolution**, while the generator outputs only **32×32** images. Despite attempts at upscaling/interpolation:

- The interpolated images lacked high-frequency content.
- The soft logits from the teacher were not meaningful.
- KD signal quality degraded significantly.

This resolution mismatch made it difficult for the student to extract useful knowledge from teacher predictions.

Ways to Improve:

To address the current shortcomings and enhance performance in future iterations, the following strategies are suggested:

1. **Better Generator Architecture**
- Use **conditional GANs** (e.g., cGANs, BigGAN) to guide class-specific image generation.
- Introduce **residual connections** or **attention blocks** in the generator to preserve feature richness.

2. **Gradient Penalty (WGAN-GP)**
- Add a **gradient penalty** term to the discriminator loss to stabilize training.
- This helps prevent discriminator overfitting and encourages smoother generator updates.
- Gradient penalty also aids in preventing mode collapse by enforcing the Lipschitz constraint in WGAN variants.

3. Stronger Distillation Losses
- Go beyond soft logit imitation—introduce:
    - **Feature-level distillation** (e.g., FitNets, AT)
    - **Adversarial feature alignment**
    - **Consistency regularization** between perturbed samples

Conclusion:

This project effectively implemented the Adversarial Knowledge Distillation (AKD) technique to

train a student model on the CIFAR-100 dataset in a data-free setting, utilizing a pre-trained ResNet-34 as the teacher. The student model, trained for 50 epochs, achieved a peak test accuracy of 1.80%.

The AKD pipeline leverages a generator to synthesize pseudo-training data and employs an L1 loss between teacher and student logits to facilitate knowledge transfer. The codebase offers a flexible framework to experiment with various ResNet-based student architectures, such as ResNet18_8x (as proposed in the original paper), enabling analysis of the trade-offs between model size and distillation performance.

A key challenge encountered was **mode collapse**, which constrained the diversity of generated samples and, consequently, the quality of knowledge transfer. As expected in zero-shot knowledge distillation (ZS-KD), a performance gap was observed between the distilled student and a conventionally trained counterpart, underscoring the difficulty of generating sufficiently informative data in the absence of real samples.

Future Works:

Future work could focus on:

- Investigating alternative ZS-KD loss functions (e.g., feature-map alignment, attention transfer, KL divergence),
- Enhancing generator design for improved sample diversity,
- Performing extensive hyperparameter tuning, and

Resources:

1. https://github.com/VainF/Data-Free-Adversarial-Distillation
2. arxiv.org/pdf/1912.11006

***